Безопасность в ASP.NET

Безопасность является одним из ключевых аспектов разработки любого API. Недостаточно реализовать корректную бизнес-логику и удобный интерфейс — необходимо также обеспечить защиту данных пользователей, ограничить доступ к ресурсам и предусмотреть защиту от типичных атак.

В данной лекции рассматриваются основные механизмы безопасности в ASP.NET Web API. Будут изучены понятия аутентификации и авторизации, структура и принцип работы JWT-токена, практическая реализация аутентификации в ASP.NET, авторизация на основе ролей и политик, безопасное хранение секретов конфигурации, а также защита от наиболее распространённых атак.

1. Аутентификация и авторизация

При проектировании системы безопасности принципиально важно различать два понятия, которые часто путают.

Аутентификация — это процесс проверки личности пользователя. Система отвечает на вопрос: кто совершает этот запрос? Типичный пример аутентификации — ввод логина и пароля, после чего система подтверждает, что пользователь является тем, за кого себя выдаёт.

Авторизация — это процесс проверки прав доступа уже аутентифицированного пользователя. Система отвечает на вопрос: что этому пользователю разрешено делать? Например, обычный пользователь может просматривать товары, но не может их удалять — это привилегия администратора.

Важно помнить, что аутентификация всегда предшествует авторизации: прежде чем проверять права доступа, система должна убедиться в том, кем является пользователь.

Подходы к аутентификации

Существует несколько распространённых подходов к аутентификации в Web API, каждый из которых имеет свои особенности.

Сессии и куки — классический подход для веб-приложений. После входа сервер создаёт сессию и сохраняет её идентификатор в куки браузера. При каждом последующем запросе сервер ищет сессию по этому идентификатору. Основной недостаток — необходимость хранить состояние на сервере, что противоречит принципу stateless, на котором построены REST API. Кроме того, такой подход плохо масштабируется: при наличии нескольких серверов сессия, созданная на одном из них, недоступна на других.

API-ключи — простой подход, при котором клиенту выдаётся уникальный ключ, передаваемый в заголовке каждого запроса. Подходит для межсервисного взаимодействия, однако не позволяет гибко управлять правами и сроком действия доступа.

Токены — наиболее предпочтительный подход для API. Клиент получает токен после успешной аутентификации и передаёт его в заголовке каждого запроса. Сервер проверяет токен и извлекает из него информацию о пользователе. Токен содержит все необходимые данные в себе, поэтому сервер не нуждается в хранении состояния. Наиболее распространённым стандартом токенов является JWT.

Структура JWT-токена

JWT (JSON Web Token) — открытый стандарт, описывающий компактный и самодостаточный способ передачи информации между сторонами в виде JSON-объекта. Токен состоит из трёх частей, разделённых точкой:

```
header.payload.signature
```

Каждая часть кодируется в формате Base64Url. Важно понимать, что Base64 — это кодирование, а не шифрование. Header и payload можно декодировать без каких-либо ключей, поэтому в токене никогда не следует хранить чувствительные данные, такие как пароли.

Header содержит метаданные токена — тип и алгоритм подписи:

```
{
  "alg": "HS256",
  "typ": "JWT"
}
```

Payload содержит claims — утверждения о пользователе и самом токене. Стандартные claims определены спецификацией:

```
{
  "sub": "42",
  "email": "user@example.com",
  "role": "Admin",
  "iat": 1720000000,
  "exp": 1720003600
}
```

Поле `sub` (subject) — идентификатор пользователя. Поле `iat` (issued at) — время выдачи токена в формате Unix timestamp. Поле `exp` (expiration) — время истечения

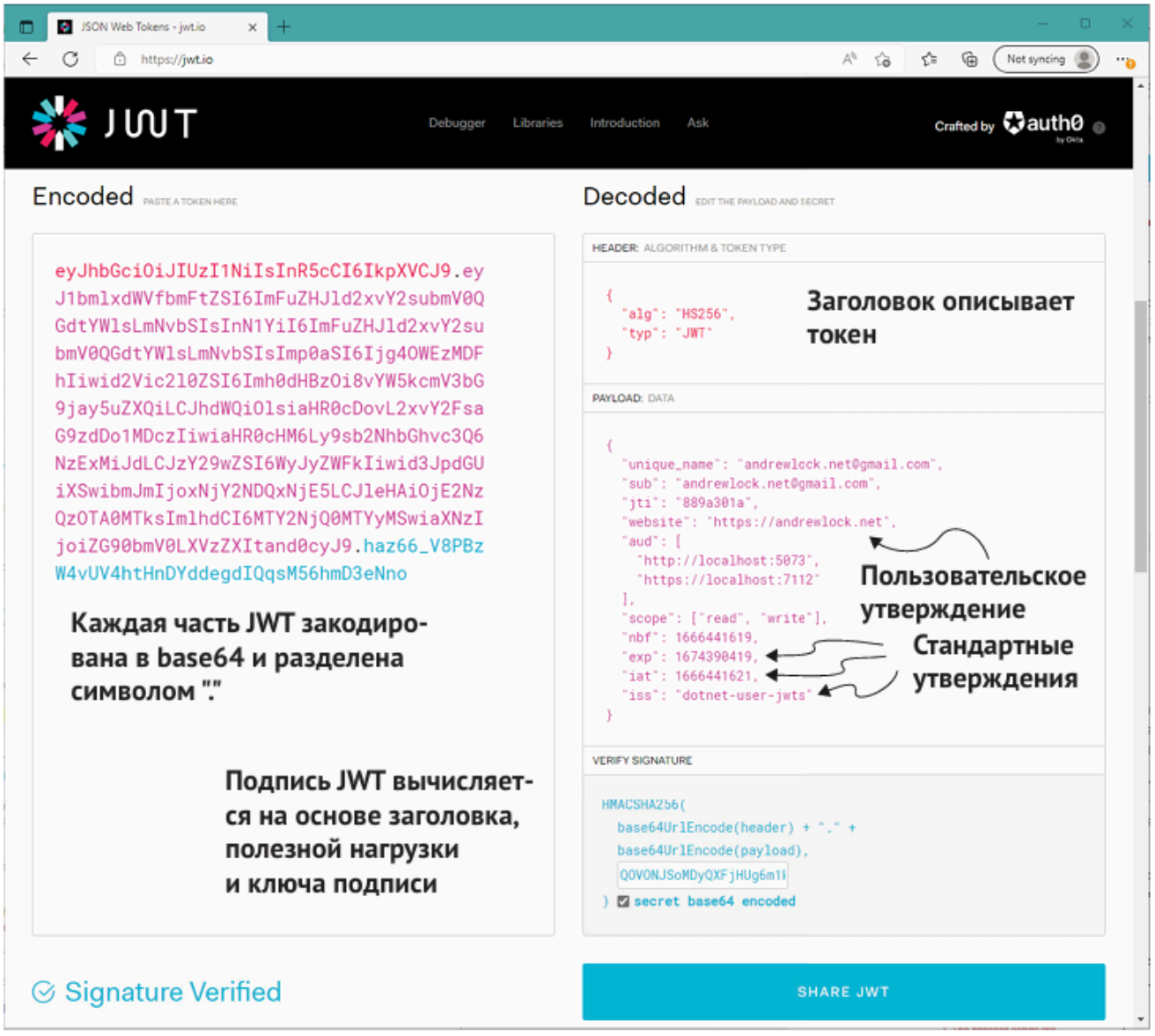
токена. Помимо стандартных, payload может содержать произвольные пользовательские claims, например `email` или `role`.

Signature — цифровая подпись, которая формируется следующим образом:

```
HMACSHA256(
    base64url(header) + "." + base64url(payload),
    secretKey
)
```

Сервер подписывает header и payload секретным ключом и шифрует с помощью алгоритма SHA256. При получении запроса сервер повторно вычисляет подпись и сравнивает её с подписью в токене. Если они совпадают — токен подлинный и не был изменён. Если злоумышленник попытается изменить payload, подпись не совпадёт и токен будет отклонён.

Для наглядного изучения структуры токена можно воспользоваться сервисом jwt.io — он позволяет декодировать токен и увидеть содержимое каждой его части.



Срок жизни токена и схема с Refresh-токеном

Поскольку JWT является самодостаточным и сервер не хранит список выданных токенов, досрочно отозвать токен стандартными средствами невозможно. Если токен был скомпрометирован, он останется действительным до истечения срока жизни. По этой причине Access-токены должны быть короткоживущими — как правило, их срок жизни составляет от 5 до 60 минут.

Однако короткий срок жизни означает, что пользователю пришлось бы часто проходить повторную аутентификацию. Для решения этой проблемы вводится схема с двумя токенами:

Access-токен — короткоживущий токен, передаваемый в заголовке каждого запроса. Используется для доступа к защищённым ресурсам.

Refresh-токен — долгоживущий токен, срок жизни которого составляет дни или недели. Используется исключительно для получения нового Access-токена после истечения его срока действия. Refresh-токен хранится в базе данных на стороне сервера, что позволяет при необходимости его отозвать.

Схема взаимодействия выглядит следующим образом:

```
1. Клиент отправляет логин и пароль
2. Сервер возвращает Access-токен и Refresh-токен
3. Клиент использует Access-токен для запросов
4. Access-токен истекает → клиент отправляет Refresh-токен
5. Сервер проверяет Refresh-токен и возвращает новый Access-токен
6. При выходе из системы Refresh-токен удаляется из БД
```

Токен передаётся в заголовке HTTP-запроса `Authorization` в формате `Bearer`:

```
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...
```

2. Реализация JWT

Подготовка проекта

Для реализации JWT-аутентификации необходимо установить следующие пакеты:

```
dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer
dotnet add package BCrypt.Net-Next
```

Пакет `Microsoft.AspNetCore.Authentication.JwtBearer` предоставляет middleware для валидации JWT-токенов. Пакет `BCrypt.Net-Next` используется для хэширования паролей.

Модель пользователя описывается следующим образом:

```
// Models/User.cs
public class User
{
    public int Id { get; set; }
    public string Email { get; set; } = string.Empty;
    public string PasswordHash { get; set; } = string.Empty;
    public string Role { get; set; } = string.Empty;
}
```

Пользователь добавляется в `AppDbContext`:

```
// Data/AppDbContext.cs
public class AppDbContext : DbContext
{
    public AppDbContext(DbContextOptions<AppDbContext> options)
        : base(options) { }

    public DbSet<Product> Products => Set<Product>();
    public DbSet<User> Users => Set<User>();
}
```

Репозиторий пользователя содержит методы, необходимые для аутентификации:

```
// Repositories/IUserRepository.cs
public interface IUserRepository
{
    Task<User?> GetByEmailAsync(string email);
    Task AddAsync(User user);
}
```

```
// Repositories/UserRepository.cs
public class UserRepository : IUserRepository
{
    private readonly AppDbContext _context;

    public UserRepository(AppDbContext context)
    {
        _context = context;
    }

    public async Task<User?> GetByEmailAsync(string email)
    {
        return await _context.Users
            .FirstOrDefaultAsync(u => u.Email == email);
    }

    public async Task AddAsync(User user)
    {
        await _context.Users.AddAsync(user);
        await _context.SaveChangesAsync();
    }
}
```

Параметры JWT выносятся в `appsettings.json`. Секретный ключ должен быть достаточно длинным — не менее 32 символов:

```
{
  "JwtSettings": {
    "SecretKey": "your-very-secret-key-at-least-32-characters",
    "Issuer": "MyApp",
    "Audience": "MyAppUsers",
    "ExpiresInMinutes": 60
  }
}
```

Для удобной работы с параметрами создаётся отдельный класс настроек:

```
// Settings/JwtSettings.cs
public class JwtSettings
{
    public string SecretKey { get; set; } = string.Empty;
    public string Issuer { get; set; } = string.Empty;
    public string Audience { get; set; } = string.Empty;
    public int ExpiresInMinutes { get; set; }
}
```

Регистрация зависимостей в `Program.cs`:

```
// Program.cs
builder.Services.Configure<JwtSettings>(
    builder.Configuration.GetSection("JwtSettings"));

builder.Services.AddScoped<IUserRepository, UserRepository>();
```

Регистрация пользователя

DTO для регистрации содержит поля email и пароль с соответствующими атрибутами валидации:

```
// DTOs/RegisterRequestDto.cs
public class RegisterRequestDto
{
    [Required(ErrorMessage = "Email обязателен для заполнения")]
    [EmailAddress(ErrorMessage = "Некорректный формат email")]
    public string Email { get; set; } = string.Empty;

    [Required(ErrorMessage = "Пароль обязателен для заполнения")]
    [MinLength(8, ErrorMessage = "Пароль должен содержать не менее 8
символов")]
    public string Password { get; set; } = string.Empty;
}
```

Контроллер аутентификации реализует endpoint регистрации. Перед сохранением пользователя выполняется проверка на уникальность email, после чего пароль хэшируется через BCrypt:

```
// Controllers/AuthController.cs
[ApiController]
[Route("api/[controller]")]
public class AuthController : ControllerBase
{
    private readonly IUserRepository _userRepository;
    private readonly IOptions<JwtSettings> _jwtSettings;

    public AuthController(
        IUserRepository userRepository,
        IOptions<JwtSettings> jwtSettings)
    {
        _userRepository = userRepository;
        _jwtSettings = jwtSettings;
    }

    [HttpPost("register")]
    public async Task<IActionResult> Register([FromBody] RegisterRequestDto
dto)
```

```
    {
        var existingUser = await _userRepository.GetByEmailAsync(dto.Email);
        if (existingUser is not null)
            return Conflict("Пользователь с таким email уже существует.");

        var user = new User
        {
            Email = dto.Email,
            PasswordHash = BCrypt.Net.BCrypt.HashPassword(dto.Password),
            Role = "User"
        };

        await _userRepository.AddAsync(user);
        return StatusCode(201, "Пользователь успешно зарегистрирован.");
    }
}
```

Метод `BCrypt.HashPassword` не просто хэширует пароль, но и автоматически генерирует и встраивает соль — случайную строку, которая добавляется к паролю перед хэшированием.

Вход и выдача токена

DTO для входа аналогичен DTO регистрации:

```
// DTOs/LoginRequestDto.cs
public class LoginRequestDto
{
    [Required(ErrorMessage = "Email обязателен для заполнения")]
    [EmailAddress(ErrorMessage = "Некорректный формат email")]
    public string Email { get; set; } = string.Empty;

    [Required(ErrorMessage = "Пароль обязателен для заполнения")]
    public string Password { get; set; } = string.Empty;
}
```

Для генерации токена создаётся отдельный вспомогательный метод. Это позволяет не дублировать логику генерации, если токен потребуется выдавать в нескольких местах, например при обновлении через Refresh-токен:

```
private string GenerateToken(User user)
{
    var settings = _jwtSettings.Value;
    var key = new SymmetricSecurityKey(
        Encoding.UTF8.GetBytes(settings.SecretKey));
    var credentials = new SigningCredentials(
        key, SecurityAlgorithms.HmacSha256);
```

```
    var claims = new[]
    {
        new Claim(ClaimTypes.NameIdentifier, user.Id.ToString()),
        new Claim(ClaimTypes.Email, user.Email),
        new Claim(ClaimTypes.Role, user.Role)
    };

    var token = new JwtSecurityToken(
        issuer: settings.Issuer,
        audience: settings.Audience,
        claims: claims,
        expires: DateTime.UtcNow.AddMinutes(settings.ExpiresInMinutes),
        signingCredentials: credentials
    );

    return new JwtSecurityTokenHandler().WriteToken(token);
}
```

Endpoint входа проверяет учётные данные и возвращает токен:

```
[HttpPost("login")]
public async Task<IActionResult> Login([FromBody] LoginRequestDto dto)
{
    var user = await _userRepository.GetByEmailAsync(dto.Email);
    if (user is null)
        return Unauthorized("Неверный email или пароль.");

    var isPasswordValid = BCrypt.Net.BCrypt.Verify(dto.Password,
user.PasswordHash);
    if (!isPasswordValid)
        return Unauthorized("Неверный email или пароль.");

    var token = GenerateToken(user);

    return Ok(new
    {
        accessToken = token,
        expiresIn = _jwtSettings.Value.ExpiresInMinutes * 60
    });
}
```

Настройка аутентификации

После реализации выдачи токена необходимо настроить ASP.NET таким образом, чтобы он валидировал токен при каждом входящем запросе. Это выполняется через `AddAuthentication` и `AddJwtBearer` в `Program.cs`:

```
// Program.cs
var jwtSettings = builder.Configuration
    .GetSection("JwtSettings")
    .Get<JwtSettings>()!;

builder.Services.AddAuthentication(options =>
{
    options.DefaultAuthenticateScheme =
JwtBearerDefaults.AuthenticationScheme;
    options.DefaultChallengeScheme = JwtBearerDefaults.AuthenticationScheme;
})
.AddJwtBearer(options =>
{
    options.TokenValidationParameters = new TokenValidationParameters
    {
        ValidateIssuer = true,
        ValidIssuer = jwtSettings.Issuer,

        ValidateAudience = true,
        ValidAudience = jwtSettings.Audience,

        ValidateLifetime = true,

        ValidateIssuerSigningKey = true,
        IssuerSigningKey = new SymmetricSecurityKey(
            Encoding.UTF8.GetBytes(jwtSettings.SecretKey))
    };
});
```

Параметр `ValidateLifetime = true` указывает, что токены с истёкшим сроком действия должны отклоняться. Параметр `ValidateIssuerSigningKey = true` обеспечивает проверку подписи токена — это ключевой параметр, без которого токен не будет проверяться на подлинность.

После настройки аутентификации необходимо подключить соответствующие middleware. Порядок их подключения принципиально важен: `UseAuthentication` должен располагаться перед `UseAuthorization`, поскольку для проверки прав доступа система сначала должна установить личность пользователя:

```
// Program.cs
app.UseAuthentication();
app.UseAuthorization();
```

Защита endpoint'ов

После настройки аутентификации можно защищать endpoint'ы с помощью атрибута `[Authorize]`. Атрибут может быть применён как к отдельному методу, так и ко всему

контроллеру:

```
[ApiController]
[Route("api/[controller]")]
[Authorize]
public class ProductsController : ControllerBase
{
    // все методы контроллера требуют аутентификации

    [HttpGet]
    public async Task<IActionResult> GetAll()
    {
        // ...
    }

    // этот метод доступен без аутентификации
    [AllowAnonymous]
    [HttpGet("public")]
    public IActionResult GetPublic()
    {
        return Ok("Этот endpoint доступен всем.");
    }
}
```

Атрибут `[AllowAnonymous]` переопределяет `[Authorize]` на уровне контроллера и делает конкретный метод доступным без токена.

Для получения данных текущего аутентифицированного пользователя из токена используется `HttpContext.User`:

```
[HttpGet("me")]
[Authorize]
public IActionResult GetCurrentUser()
{
    var userId = User.FindFirst(ClaimTypes.NameIdentifier)?.Value;
    var email = User.FindFirst(ClaimTypes.Email)?.Value;
    var role = User.FindFirst(ClaimTypes.Role)?.Value;

    return Ok(new { userId, email, role });
}
```

ASP.NET автоматически извлекает claims из валидного токена и помещает их в `HttpContext.User`, делая их доступными в любом методе контроллера.

3. Авторизация на основе ролей и политик

Авторизация на основе ролей

Авторизация на основе ролей — наиболее распространённый способ разграничения доступа. Идея проста: каждому пользователю назначается роль, и доступ к определённым ресурсам предоставляется только пользователям с нужной ролью.

В предыдущем разделе при генерации токена в payload добавлялся claim `ClaimTypes.Role`. Именно на основании этого claim ASP.NET определяет роль пользователя при авторизации. Для ограничения доступа по роли используется параметр `Roles` атрибута `[Authorize]`:

```
[ApiController]
[Route("api/[controller]")]
public class ProductsController : ControllerBase
{
    // доступно всем аутентифицированным пользователям
    [HttpGet]
    [Authorize]
    public async Task<IActionResult> GetAll()
    {
        // ...
    }

    // доступно только администраторам
    [HttpDelete("{id}")]
    [Authorize(Roles = "Admin")]
    public async Task<IActionResult> Delete(int id)
    {
        // ...
    }

    // доступно нескольким ролям
    [HttpPost]
    [Authorize(Roles = "Admin,Manager")]
    public async Task<IActionResult> Create([FromBody] ProductRequestDto
dto)
    {
        // ...
    }
}
```

Если пользователь аутентифицирован, но его роль не совпадает с требуемой, ASP.NET вернёт ответ `403 Forbidden`. Если пользователь не аутентифицирован вовсе, будет возвращён ответ `401 Unauthorized`.

Ролевая модель хорошо работает в простых случаях, однако имеет существенное ограничение: она не позволяет выразить сложные правила доступа. Например, правило «пользователь может редактировать только свои собственные записи» не описывается через роль — здесь требуется более гибкий механизм.

Авторизация на основе политик

Политики — более гибкий механизм авторизации, позволяющий описывать произвольные условия доступа на основе claims пользователя или любой другой логики. Политики регистрируются в `Program.cs` и затем применяются через атрибут `[Authorize(Policy = "...")]`.

Рассмотрим несколько практических примеров.

Пример 1: политика на основе роли. На первый взгляд это дублирует ролевую авторизацию, однако политика позволяет в будущем расширить условие без изменения атрибутов на контроллерах:

```
// Program.cs
builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("AdminOnly", policy =>
        policy.RequireRole("Admin"));
});
```

Пример 2: политика на основе claim. Доступ предоставляется только пользователям, у которых в токене присутствует определённый claim с определённым значением:

```
builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("AdminOnly", policy =>
        policy.RequireRole("Admin"));

    options.AddPolicy("VerifiedUser", policy =>
        policy.RequireClaim("email_verified", "true"));
});
```

Пример 3: комбинированная политика. Условия можно комбинировать — пользователь должен одновременно соответствовать нескольким требованиям:

```
builder.Services.AddAuthorization(options =>
{
    options.AddPolicy("AdminOnly", policy =>
        policy.RequireRole("Admin"));

    options.AddPolicy("VerifiedUser", policy =>
```

```
        policy.RequireClaim("email_verified", "true"));

    options.AddPolicy("VerifiedAdmin", policy =>
    {
        policy.RequireRole("Admin");
        policy.RequireClaim("email_verified", "true");
    });
});
```

Применение политики на контроллере выглядит следующим образом:

```
[HttpDelete("{id}")]
[Authorize(Policy = "AdminOnly")]
public async Task<IActionResult> Delete(int id)
{
    // ...
}
```

4. Безопасное хранение секретов

В предыдущих разделах секретный ключ JWT и строка подключения к базе данных хранились в файле `appsettings.json`. Для учебных целей это приемлемо, однако в реальных проектах такой подход недопустим.

Основная опасность заключается в том, что `appsettings.json` как правило добавляется в систему контроля версий. Это означает, что секретные данные становятся доступны всем, кто имеет доступ к репозиторию. Даже если файл был добавлен в `.gitignore` после того, как однажды попал в репозиторий, он остаётся в истории коммитов и может быть извлечён оттуда. Утечка секретного ключа JWT означает, что злоумышленник сможет самостоятельно подписывать токены и получать доступ к любому аккаунту в системе.

Таким образом, правило простое: `appsettings.json` не должен содержать никаких секретных данных — ни ключей, ни паролей, ни строк подключения с учётными данными.

User Secrets

User Secrets — встроенный механизм ASP.NET, предназначенный для хранения секретов в процессе локальной разработки. Секреты хранятся за пределами папки проекта, в специальной директории операционной системы, и поэтому никогда не попадают в репозиторий.

На Windows секреты хранятся по следующему пути:

```
%APPDATA%\Microsoft\UserSecrets\<user-secrets-id>\secrets.json
```

На Linux и macOS:

```
~/.microsoft/usersecrets/<user-secrets-id>/secrets.json
```

Для инициализации User Secrets в проекте выполняется следующая команда:

```
dotnet user-secrets init
```

После выполнения команды в файл `.csproj` добавляется уникальный идентификатор:

```
<PropertyGroup>
    <UserSecretsId>your-unique-guid-here</UserSecretsId>
</PropertyGroup>
```

Добавление секретов выполняется через команду `dotnet user-secrets set`. Для переопределения вложенных параметров конфигурации используется двоеточие в качестве разделителя:

```
dotnet user-secrets set "JwtSettings:SecretKey" "my-super-secret-key-for-development"
dotnet user-secrets set "ConnectionStrings:DefaultConnection" "Data Source=app.db"
```

Просмотр всех сохранённых секретов:

```
dotnet user-secrets list
```

Удаление конкретного секрета:

```
dotnet user-secrets remove "JwtSettings:SecretKey"
```

Ключевое преимущество User Secrets состоит в том, что доступ к ним осуществляется через стандартный интерфейс `IConfiguration` — тот же, что используется для чтения `appsettings.json`. Никаких изменений в коде не требуется:

```
// Этот код работает одинаково как для appsettings.json, так и для User Secrets
var jwtSettings = builder.Configuration
    .GetSection("JwtSettings")
    .Get<JwtSettings>()!;
```

ASP.NET автоматически подключает User Secrets в режиме разработки (`Development`). Если проект запускается в режиме `Production`, User Secrets не загружаются — это намеренное поведение, поскольку для рабочей версии используются другие механизмы.

Важно понимать, что User Secrets не шифруются — они хранятся в виде обычного JSON-файла. Их безопасность обеспечивается исключительно тем, что файл находится за пределами репозитория и доступен только текущему пользователю операционной системы.

Переменные окружения

Переменные окружения являются стандартным способом передачи секретов в продакшен-окружении. Они устанавливаются на уровне операционной системы или платформы развёртывания и недоступны из кода репозитория.

ASP.NET автоматически читает переменные окружения и включает их в `IConfiguration`. Для переопределения вложенных параметров конфигурации

используется двоеточие в качестве разделителя на Windows и двойное подчёркивание на Linux:

```
# Windows
set JwtSettings__SecretKey=my-production-secret-key
set ConnectionStrings__DefaultConnection=Data Source=/var/app/production.db

# Linux / macOS
export JwtSettings__SecretKey=my-production-secret-key
export ConnectionStrings__DefaultConnection=Data
Source=/var/app/production.db
```

Приоритет источников конфигурации

ASP.NET загружает конфигурацию из нескольких источников, применяя их последовательно. Каждый последующий источник переопределяет значения предыдущего. Порядок приоритетов от низшего к высшему выглядит следующим образом:

```
appsettings.json
        ↓
appsettings.{Environment}.json
        ↓
User Secrets (только в режиме Development)
        ↓
Переменные окружения
        ↓
Аргументы командной строки
```

Это означает, что переменная окружения всегда переопределит значение из `appsettings.json`, а аргумент командной строки переопределит переменную окружения. Данная иерархия позволяет хранить значения по умолчанию в `appsettings.json`, не опасаясь, что они будут использованы в продакшене, где те же параметры задаются через переменные окружения.

Таким образом, файл `appsettings.json` может выглядеть следующим образом — с явно обозначенными заглушками вместо реальных значений:

```
{
  "ConnectionStrings": {
    "DefaultConnection": "placeholder"
  },
  "JwtSettings": {
    "SecretKey": "placeholder",
    "Issuer": "MyApp",
    "Audience": "MyAppUsers",
```

```
    "ExpiresInMinutes": 60
  }
}
```

Реальные значения `SecretKey` и `DefaultConnection` будут переопределены из User Secrets при локальной разработке и из переменных окружения в продакшене.

5. Защита от типичных атак

HTTPS и принудительное перенаправление

HTTPS — это протокол HTTP с шифрованием трафика посредством TLS. Без HTTPS все данные, передаваемые между клиентом и сервером, включая JWT-токены и пароли, передаются в открытом виде и могут быть перехвачены злоумышленником в рамках атаки типа Man-in-the-Middle (человек посередине).

В ASP.NET принудительное перенаправление с HTTP на HTTPS подключается одной строкой:

```
// Program.cs
app.UseHttpsRedirection();
```

После подключения этого middleware любой запрос, пришедший по HTTP, будет автоматически перенаправлен на HTTPS с кодом `301 Moved Permanently`.

CORS

CORS (Cross-Origin Resource Sharing) — механизм безопасности браузера, который блокирует запросы к API с домена, отличного от домена самого API. Например, если фронтенд расположен на `https://myfrontend.com`, а API на `https://myapi.com`, браузер заблокирует такой запрос без явного разрешения со стороны сервера.

Важно понимать, что CORS — это ограничение браузера, а не сервера. Запросы от серверных приложений, утилит типа curl или Postman не подвержены этому ограничению. Цель CORS — защита пользователей от вредоносных сайтов, которые могут выполнять запросы к API от их имени.

Настройка CORS в ASP.NET выполняется в `Program.cs`. Сначала определяется политика, затем она применяется через middleware:

```
// Program.cs
builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowFrontend", policy =>
    {
        policy.WithOrigins("https://myfrontend.com")
              .WithMethods("GET", "POST", "PUT", "DELETE")
              .WithHeaders("Authorization", "Content-Type");
    });
});
```

```
// Program.cs
app.UseCors("AllowFrontend");
```

Порядок подключения middleware имеет значение — `UseCors` должен располагаться до `UseAuthentication` и `UseAuthorization`.

Распространённая ошибка при настройке CORS — использование `AllowAnyOrigin` в рабочей версии программы:

```
// Небезопасно для продакшена
policy.AllowAnyOrigin()
      .AllowAnyMethod()
      .AllowAnyHeader();
```

Такая настройка разрешает запросы с любого домена, что открывает возможность для CSRF-атак. **CSRF (Cross-Site Request Forgery)** — атака, при которой вредоносный сайт вынуждает браузер аутентифицированного пользователя отправить запрос к API без его ведома. Например, пользователь авторизован в банковском приложении, затем переходит на вредоносный сайт, который в фоне отправляет запрос на перевод средств от его имени. Браузер автоматически прикладывает к запросу куки пользователя, и сервер воспринимает его как легитимный.

Именно поэтому настройка `AllowAnyOrigin` опасна: она снимает одно из препятствий для подобных атак, разрешая запросы с любого домена, включая вредоносные.

Если необходимо разрешить несколько доменов, они перечисляются явно через `WithOrigins`. При явном указании допустимых доменов браузер заблокирует запросы со всех остальных источников, что существенно затрудняет проведение CSRF-атаки:

```
policy.WithOrigins(
        "https://myfrontend.com",
        "https://admin.myfrontend.com")
      .WithMethods("GET", "POST", "PUT", "DELETE")
      .WithHeaders("Authorization", "Content-Type");
```

SQL-инъекции

SQL-инъекция — атака, при которой злоумышленник внедряет произвольный SQL-код в запрос через входные данные. Классический пример: если приложение формирует запрос конкатенацией строк, передача значения `'; DROP TABLE Users; --` в качестве параметра может привести к удалению таблицы.

Entity Framework Core защищает от SQL-инъекций автоматически, поскольку все запросы через LINQ транслируются в параметризованные SQL-запросы.

Параметризация означает, что значения передаются отдельно от текста запроса и никогда не интерпретируются как SQL-код:

```
// Безопасно — EF Core генерирует параметризованный запрос
var user = await _context.Users
    .FirstOrDefaultAsync(u => u.Email == email);

// SQL, который генерирует EF Core:
// SELECT * FROM Users WHERE Email = @p0
// @p0 = 'user@example.com'
```

Опасность возникает при использовании сырых SQL-запросов через `FromSqlRaw` с конкатенацией строк:

```
// Опасно — уязвимость к SQL-инъекции
var users = await _context.Users
    .FromSqlRaw($"SELECT * FROM Users WHERE Email = '{email}'")
    .ToListAsync();
```

Если переменная `email` содержит `' OR '1'='1`, запрос вернёт всех пользователей. Если она содержит `'; DROP TABLE Users; --`, последствия могут быть катастрофическими.

При необходимости использовать сырые SQL-запросы следует применять параметризованный вариант:

```
// Безопасно — значение передаётся как параметр
var users = await _context.Users
    .FromSqlRaw("SELECT * FROM Users WHERE Email = {0}", email)
    .ToListAsync();
```

Либо использовать `FromSqlInterpolated`, который автоматически обрабатывает интерполированные строки как параметры:

```
// Безопасно — FromSqlInterpolated автоматически параметризует значения
var users = await _context.Users
    .FromSqlInterpolated($"SELECT * FROM Users WHERE Email = {email}")
    .ToListAsync();
```

Mass Assignment

Mass Assignment — уязвимость, при которой клиент может изменить поля объекта, которые он не должен иметь возможности изменять. Типичный пример: модель пользователя содержит поле `IsAdmin`. Если контроллер принимает объект напрямую

без использования DTO, злоумышленник может передать `"isAdmin": true` в теле запроса и получить права администратора.

Рассмотрим пример уязвимого кода:

```
// Опасно — принимается доменная модель напрямую
[HttpPut("{id}")]
public async Task<IActionResult> Update(int id, [FromBody] User user)
{
    // Клиент может передать любое поле модели User,
    // включая IsAdmin, PasswordHash и другие
    _context.Users.Update(user);
    await _context.SaveChangesAsync();
    return NoContent();
}
```

Использование DTO полностью устраняет эту уязвимость, поскольку контроллер принимает только те поля, которые явно описаны в DTO:

```
// DTOs/UpdateUserRequestDto.cs
public class UpdateUserRequestDto
{
    [Required]
    [MaxLength(100)]
    public string Name { get; set; } = string.Empty;

    [EmailAddress]
    public string Email { get; set; } = string.Empty;

    // Поля IsAdmin, PasswordHash и Role здесь отсутствуют —
    // клиент не может их изменить
}
```

```
// Безопасно — принимается только то, что описано в DTO
[HttpPut("{id}")]
public async Task<IActionResult> Update(
    int id,
    [FromBody] UpdateUserRequestDto dto,
    [FromServices] IUserRepository repository)
{
    var user = await repository.GetByIdAsync(id);
    if (user is null)
        return NotFound();

    _mapper.Map(dto, user);
    await repository.UpdateAsync(user);
```

```
    return NoContent();
}
```

Таким образом, DTO выполняют двойную функцию: они не только формируют удобный контракт API, но и служат барьером, предотвращающим несанкционированное изменение данных.

Rate Limiting

Rate Limiting — механизм ограничения количества запросов от одного клиента за определённый промежуток времени. Он защищает API от нескольких видов угроз: брутфорс-атак на endpoint аутентификации, чрезмерной нагрузки на сервер и автоматизированного перебора данных.

В ASP.NET встроен middleware для Rate Limiting, не требующий установки дополнительных пакетов. Существует несколько стратегий ограничений.

Фиксированное окно (Fixed Window) — разрешает не более N запросов за фиксированный период времени. По истечении периода счётчик сбрасывается:

```
// Program.cs
builder.Services.AddRateLimiter(options =>
{
    options.AddFixedWindowLimiter("fixed", limiterOptions =>
    {
        limiterOptions.PermitLimit = 10;
        limiterOptions.Window = TimeSpan.FromMinutes(1);
        limiterOptions.QueueProcessingOrder =
QueueProcessingOrder.OldestFirst;
        limiterOptions.QueueLimit = 0;
    });

    // Код ответа при превышении лимита
    options.RejectionStatusCode = 429;
});
```

Скользящее окно (Sliding Window) — аналогично фиксированному окну, но период сдвигается вместе с текущим временем. Это позволяет более равномерно распределить нагрузку и исключить ситуацию, когда клиент отправляет все N запросов в конце одного периода и N запросов в начале следующего:

```
builder.Services.AddRateLimiter(options =>
{
    options.AddSlidingWindowLimiter("sliding", limiterOptions =>
    {
        limiterOptions.PermitLimit = 10;
        limiterOptions.Window = TimeSpan.FromMinutes(1);
```

```
        limiterOptions.SegmentsPerWindow = 6;
        limiterOptions.QueueProcessingOrder =
QueueProcessingOrder.OldestFirst;
        limiterOptions.QueueLimit = 0;
    });

    options.RejectionStatusCode = 429;
});
```

Middleware подключается в `Program.cs`:

```
app.UseRateLimiter();
```

Применение ограничения к конкретному endpoint осуществляется через атрибут `[EnableRateLimiting]`. Особенно важно применять Rate Limiting к endpoint'ам аутентификации, поскольку именно они являются первоочередной целью брутфорс-атак:

```
[HttpPost("login")]
[EnableRateLimiting("fixed")]
public async Task<IActionResult> Login([FromBody] LoginRequestDto dto)
{
    // ...
}
```

Ограничение можно также применить ко всему контроллеру:

```
[ApiController]
[Route("api/[controller]")]
[EnableRateLimiting("fixed")]
public class AuthController : ControllerBase
{
    // все методы контроллера подпадают под ограничение
}
```

Если Rate Limiting применён глобально и нужно исключить из него конкретный endpoint, используется атрибут `[DisableRateLimiting]`:

```
[HttpGet("health")]
[DisableRateLimiting]
public IActionResult HealthCheck()
{
    return Ok("Healthy");
}
```

При превышении лимита запросов сервер возвращает ответ `429 Too Many Requests`. Клиент должен обрабатывать этот статус и выполнять повторный запрос с задержкой.